

Optimal Resizable Arrays 论文评述

张艺博 徐黄浩

2026 年 7 月 13 日

摘要

本文评述 Robert E. Tarjan 与 Uri Zwick 的论文 *Optimal resizable arrays*。该论文研究如何在保持数组随机访问效率的同时，降低动态扩缩容带来的额外空间开销。本文将围绕问题模型、相关工作、核心数据结构、复杂度证明以及方法局限展开分析。

目录

1	引言与论文定位	4
1.1	成员 A: 问题直觉与论文定位	4
1.2	成员 B: 补充与交叉审阅	5
2	问题定义与理论模型	5
2.1	成员 A: 补充与交叉审阅	5
2.2	成员 B: 问题定义与理论模型	5
2.2.1	可变长数组的抽象接口	5
2.2.2	内存块模型	6
2.2.3	两类空间开销	6
2.2.4	本文的复杂度目标	7
3	文献调研与相关工作	7
3.1	成员 A: 传统方法与工程背景	7
3.2	传统工作	8
3.2.1	摊还分析背景	8
3.2.2	其他传统工作	8
3.3	成员 B: 理论脉络与下界背景	9
3.3.1	Brodnik 等人的空间下界	9
3.3.2	Tarjan-Zwick 对下界视角的扩展	10
4	本文核心方法	11
4.1	成员 A: 整体构造与技术路线	11
4.2	整体构造与技术路线: 从空间下界到构造目标	11
4.3	成员 B: 空间优化机制	13
4.3.1	$r = 3$ 的两级块结构	13
4.3.2	一般 r 的分层块结构	14
4.3.3	访问时间与均摊更新时间	14
4.3.4	去除稳定空间中的 r 因子	15
5	正确性证明与理论分析	16
5.1	成员 A: 上界分析	16
5.2	成员 B: 下界分析	16

6	技术直觉与方法评价	16
6.1	成员 A：技术直觉与示例解释	16
6.2	成员 B：创新点、局限性与后续影响	16
7	总结	16
7.1	成员 A：总结	16
7.2	成员 B：参考文献与过程说明	16

1 引言与论文定位

1.1 成员 A：问题直觉与论文定位

数组和可变数组是广泛应用的数据结构，可变数组的内存效率一直是研究者的研究热点。本文所研究的可变数组指的是，可以在末尾添加或者删除元素的数组[1]。为此，很自然想到，如何解决可变长数组的空间分配问题？一个典型场景是：内存已分配的空间已经被填满，此时需要插入新元素，却不能直接在原有内存块末尾追加，因为紧邻的内存可能已被其他对象占用。因此，必须重新申请一块更大的内存，将原有元素全部复制过去，再释放旧块。

也就是：如何维护一个可以增长和收缩的数组，使它既支持按下标 $O(1)$ 访问，又尽可能节省空间，同时 grow/shrink 操作仍然高效？

此问题在工程中与实际中用处广泛，比如：Java ArrayList，C++ vector。但是，作者认为，之前的设计（满了扩容、空了减小）过于基础，尽管它是最常用的数据结构之一，但是它的空间最优性没有完全解决。

这一问题可以从**机制**与**策略**两个层面加以分离与理解。

机制层面

1. 系统提供固定大小的存储块（block）；
2. 存储块可以存放实际数据，也可以存放指向其他块的指针；
3. Grow 操作：申请新块，必要时进行数据迁移；
4. Shrink 操作：删除末尾元素，释放不再需要的空块。

策略层面

1. 块的大小如何选取？
2. 使用多少层指针？Grow 过程中块的大小如何变化？
3. Shrink 的触发条件是什么？何时进行全局重建（rebuild）？

将机制与策略解耦，有助于清晰地分析和设计可变长数组的数据结构。

经典方法：翻倍策略（Doubling） 工业界广泛采用的成熟做法是几何增长策略，其核心规则为：当数组满时，分配一块大小为当前数组两倍的连续内存，并将所有元素复制过去。这一策略的摊还代价为 $O(1)$ ，随机访问同样为 $O(1)$ 最坏情况时间[2]。标准倍增算法可以达到 $O(N)$ （更确切说，应该是 $N + O(N)$ ）空间开销和 $O(1)$ 的时间开

销，但其最坏时刻的空间浪费可达 50%。进一步，我们可以将这些策略总结成如下规律(再后面第三节传统文献调研部分有详细说明)：假设每次数组满后，增长因子为 $(1 + \alpha)$ ，那么新数组刚扩展完毕时浪费率为 $\frac{\alpha}{1+\alpha}$ ，但是， α 也不能无限小，当其越小时，时间开销越大，因为平均复制次数增多了，假设数组容量为 C 后触发 grow，新容量变为 $C' = (1 + \alpha)C$ ，每个操作的成本为 $1/\alpha$

本文的目标 能否将空间占用压缩至 $N + o(N)$ ，同时仍然保持 $O(1)$ 的随机访问时间？这正是 Tarjan 和 Zwick 论文试图回答的核心问题[1]。

核心思想 作者认为应该区分两种空间，第一种是存储空间：数组当前大小为 N ，平时为了存储它并支持访问，需要多少空间？第二种是resize 临时空间：grow/shrink 的过程中，短暂需要多少额外空间？

1.2 成员 B：补充与交叉审阅

待补充。

2 问题定义与理论模型

2.1 成员 A：补充与交叉审阅

待补充。

2.2 成员 B：问题定义与理论模型

本节给出本文所讨论的可变长数组（resizable array）的形式化定义，并说明 Tarjan 和 Zwick 论文中用于衡量空间开销的模型[1]。与普通动态序列不同，本文研究的对象更接近栈式数组：元素只能在数组末端增加或删除，但任意位置的读取与修改仍然需要保持常数时间。

2.2.1 可变长数组的抽象接口

设当前数组为 A ，长度为 N 。论文中的可变长数组支持以下基本操作：

1. `Array()`：创建并返回一个初始为空的数组；
2. `Length()`：返回数组当前长度；
3. `Get(i)`：返回下标为 i 的元素，其中 $0 \leq i < N$ ；

4. **Set(i, a)**: 将下标为 i 的元素修改为 a , 其中 $0 \leq i < N$;
5. **Grow(a)**: 在数组末尾加入一个新元素 a , 使数组长度从 N 变为 $N + 1$;
6. **Shrink()**: 删除数组末尾元素, 使数组长度从 N 变为 $N - 1$ 。

这里最重要的限制是：插入和删除只发生在末端。若允许在任意位置插入或删除，则后续元素的下标都可能发生变化，问题会转化为动态序列维护。该类问题即使不要求极端空间效率，也通常需要更复杂的数据结构和更高的操作复杂度。因此，本文刻意将研究对象限定在末端扩缩容的数组模型上。

2.2.2 内存块模型

论文假设底层内存管理系统允许申请和释放任意固定长度的数组块。一个可变长数组的具体实现由若干动态分配的块组成，主要包括三类：

1. **数据块 (data block)**: 存放数组中的实际元素；
2. **索引块 (index block)**: 存放指向数据块或其他索引块的指针；
3. **辅助块 (auxiliary block)**: 存放块数量、块长度、层级计数器等辅助信息。

这种模型抽象了实际系统中的动态内存分配行为。若数组必须始终存放在单个连续内存块中，那么扩容时通常只能重新申请更大的块并复制所有元素；而本文允许数组被分散存放在多个块中，再通过索引块维持常数时间访问。这也是后续分层结构能够降低空间浪费的基础。

2.2.3 两类空间开销

本文的关键建模创新是区分两类空间：

1. **存储与访问空间**: 当数组处于稳定状态、长度为 N 时，为了存储所有元素并支持 **Get** 和 **Set** 所需的空間；
2. **扩缩容临时空间**: 执行 **Grow** 或 **Shrink** 时，在元素移动、块合并或块拆分过程中短暂额外占用的空间。

用论文中的记号表示，若一个实现称为 $(s(N), t(N))$ -implementation，则表示它满足[1]：

$$\text{稳定存储空间} \leq N + s(N),$$

并且在扩缩容操作过程中最多使用：

$$\text{临时峰值空间} \leq N + t(N).$$

这里的 $s(N)$ 衡量平时多出来的空间，例如索引指针、空闲块和辅助信息； $t(N)$ 衡量一次 grow 或 shrink 过程中可能出现的峰值空间。传统讨论往往把这两种空间混在一起计算，而 Tarjan 和 Zwick 的核心观察是：在很多应用中，平时占用空间比扩缩容瞬间的临时空间更值得优化。例如，一个程序可能同时维护许多可变长数组，但任意时刻只有少数数组正在扩容或缩容。

2.2.4 本文的复杂度目标

在上述模型下，本文希望同时达到以下目标：

1. Get 和 Set 操作保持 $O(1)$ 最坏情况时间；
2. Grow 和 Shrink 操作保持较低的均摊时间；
3. 稳定存储空间尽量接近信息论意义上的下限 N ；
4. 在允许一定临时空间的前提下，突破传统 $N + O(\sqrt{N})$ 空间界限。

论文最终给出的结果是一条由参数 r 控制的折中曲线：对于任意整数 $r \geq 2$ ，可以将稳定存储空间压到 $N + O(N^{1/r})$ ，同时扩缩容时允许短暂使用 $N + O(N^{1-1/r})$ 空间，并保持访问 $O(1)$ 最坏情况时间、扩缩容 $O(r)$ 均摊时间。后文对核心结构和下界证明的分析，都是围绕这组时间-空间折中展开的。

3 文献调研与相关工作

3.1 成员 A：传统方法与工程背景

在进行相关工作阐述之前，我们先解释一下摊还分析(Amortized Analysis)。

摊还分析是一种分析操作序列平均代价的技术，由于难以衡量单次操作的最坏代价，所以我们从总代价的角度出发，使用总代价除以操作次数，得到平均每次操作花费代价。

具体而言，有三种分析方法：

1. 聚合分析：直接计算总代价，除以操作次数
2. 记账法
3. 势能法

本文中使用的主要是第一种方法。

3.2 传统工作

首先介绍论文中提到的相关工作

朴素方法 很自然想到，最直接实现可变长数组的方案就是维护一个大小恰好等于当前元素数 N 的固定数组，每次 grow 或者 shrink 时都触发全量重建

它的存储空间显然是 $N + O(1)$ ，可以达到理论最优，但是 grow 或者 shrink 的摊还代价为 $\Theta(N)$ ，并且临时空间最大为 $2N + O(1)$

Geometric Expansion 第二种传统方法就是前文提到过的几何增长策略，用空间换取时间，通过预分配额外容量降低重建成本。它的具体方法如下：

假设增长因子为 $1 + \alpha$ ，初始分配大小为 C 的数组，当 N 个元素填满后，分配大小为 $(1 + \alpha)N$ 的新数组，复制全部元素，当元素数降至 $\frac{N}{1+\alpha}$ 以下时，收缩至 $\frac{N}{1+\alpha}$

使用摊还分析该数据结构，它的存储空间为 $O(N)$ ，grow 摊还代价为 $N/(\alpha N) = 1/\alpha$

可以看到，如果我们增大 α ，虽然摊还代价小了，但是我们总的存储空间增加，因此，传统方法的局限就在于此，难以平衡空间和时间。根本原因在于粒度太粗，也就是现在的空间划分只是计算的平均空间，后续作者针对这种不足提出了本文的改进。

3.2.1 摊还分析背景

在进行相关工作阐述之前，我们先解释一下摊还分析(Amortized Analysis)。

摊还分析是一种分析操作序列平均代价的技术，由于难以衡量单次操作的最坏代价，所以我们从总代价的角度出发，使用总代价除以操作次数，得到平均每次操作花费代价。

具体而言，有三种分析方法：

1. 聚合分析：直接计算总代价，除以操作次数 2. 记账法 3. 势能法

本文中使用的主要是第一种方法。

3.2.2 其他传统工作

这里介绍本文中未提及的其他工作

实际上，从工程角度出发，很多语言都有自己的方法，但是绝大部分都是遵循的前面提到的第二种方法：几何增长策略，只不过增长因子不同。例如：C++ `std::vector` 使用 $\alpha = 2$ ，Java `ArrayList` 使用 $\alpha = 1.5$ ，python `list` 使用 1.125。这里有一个非常有意思的点，就是Java中选择增长因子为1.5。我们考虑一个关键的工程因素，也就是释放后的旧块未来能否被新块利用？如果说旧块累积总容量超过了需要的新块，那么内存分配器就可以分配，否则，内存很容易满（更容易理解的说法是如果我下一次只需要临时用一

点内存，但是分配的新块如果很大，结果没有足够的内存，但实际上我们只需要分配一点内存就能满足，这说明 α 不能过大。

此外，链表与分块链表也是解决动态数组的一种策略，考虑一般的分块链表，每个节点为一个大小为 B 的数组块，虽然其能够做到空间 $N + O(N/B)$ ，但是因为它无法做到随机访问（访问时间为 $O(N)$ ），不能成为动态数组的替代。

Phil等人提出的VList对上面的分块链表做了改进，将每个节点的Block大小设置为可变的，大小按照几何比例变化（1,2,4,8...），这其实是一个很好的平衡，它使得访问时间变为 $O(\log N)$ ，但是最坏存储空间浪费为 $2N + O(\log N)$ （这是因为刚扩容的时候申请的空闲块大小为前面的两倍），这是其不足的一点，此外，

Bagwell 等人提出的 VArray 继续延伸 VList 思想，VArray 采用分块层级结构，当前层填满后，新建更大的block层，并通过指针连接旧层，从而避免 resize 时的开销。其优势是 grow 操作不需要整体搬移已有元素；代价是数组不再是一整块连续内存，随机访问通常需要额外间接寻址。

Gap Buffer 是文本编辑器中经典的动态数组变体，常见于 GNU Emacs 等文本编辑系统。其基本动机来自文本编辑的局部性：用户在编辑文本时，通常会在当前光标附近连续插入、删除字符，而不是只在序列末尾追加或删除元素。若直接使用普通动态数组表示文本，则在中间位置插入字符往往需要移动插入点之后的大量元素，代价较高。Gap Buffer 的核心思想是在当前光标位置附近维护一段未使用的连续空洞（gap）：插入操作通过填充该空洞完成，删除操作则通过扩大空洞完成，因此光标附近的连续编辑可以高效执行。

3.3 成员 B：理论脉络与下界背景

本节梳理可变长数组空间效率问题的前置工作。传统动态数组通过几何扩容获得 $O(1)$ 均摊更新时间，但会保留线性级别的空闲容量[2]；Sitariski 的 hashed array tree 和 Brodник 等人的结构则把稳定状态下的额外空间降低到 $O(\sqrt{N})$ [3, 4]。Tarjan 和 Zwick 的工作建立在这条脉络上，但其关键贡献不是简单改进已有结构，而是重新区分“稳定存储与访问空间”和“扩缩容临时空间”[1]。

3.3.1 Brodник 等人的空间下界

Brodник 等人的经典结果说明，在不区分临时空间的模型中， $N + \Omega(\sqrt{N})$ 级别的空间占用是不可避免的[4]。这个下界的证明思路很简洁，也解释了为什么后续工作很难在原有模型下突破 $\sqrt{N}$ 壁垒。

考虑只执行 N 次 Grow 操作后的状态。设此时数组元素分布在 k 个连续内存块中。由于这些块共同保存 N 个元素，它们的总大小至少为 N 。同时，为了之后能够访问每

个块，数据结构至少需要维护到这些块的指针或等价索引信息。

如果 $k \geq \sqrt{N}$ ，那么仅指针数量就已经造成 $\Omega(\sqrt{N})$ 的额外空间。因此总空间至少为

$$N + \Omega(\sqrt{N}).$$

如果 $k < \sqrt{N}$ ，那么在保存 N 个元素的这些块中，必然存在一个大小超过 $\sqrt{N}$ 的块。设这个大块是在数组长度为 $N' \leq N$ 时被分配的。它刚被分配出来时还没有装入原有元素，而旧的 N' 个元素仍然存放在其他地方，所以当时的总空间会超过

$$N' + \sqrt{N'}.$$

因此，无论块数多还是少，某个时刻都会出现 $N + \Omega(\sqrt{N})$ 级别的空间占用。直观地说，块多会带来指针开销，块少则会带来大块分配时的临时空间峰值。

3.3.2 Tarjan–Zwick 对下界视角的扩展

Tarjan 和 Zwick 并不是否定 Brodник 等人的下界，而是指出该下界把两类空间混在了一起：一类是平时存储和访问数组需要的空间，另一类是执行 **Grow** 或 **Shrink** 时短暂出现的临时空间[1]。若用 $(s(N), t(N))$ -implementation 描述二者，则平时总空间为 $N + s(N)$ ，扩缩容过程中的峰值空间为 $N + t(N)$ 。

在这个模型下，Brodник 下界可以推广为乘积形式。仍然考虑执行 N 次 **Grow** 后的状态。由于每一个数据块都必须可达，块数不能超过额外索引空间的量级，因此可以认为块数至多为 $O(s(N))$ 。这些块一共存放 N 个元素，所以至少有一个块的大小为

$$\Omega\left(\frac{N}{s(N)}\right).$$

当这个大块刚被分配出来时，它尚未存放旧元素，旧元素仍然需要由原有块保存。因此在那次 **Grow** 的过程中，临时额外空间至少达到该大块的大小量级，即

$$t(N) = \Omega\left(\frac{N}{s(N)}\right).$$

于是得到

$$s(N)t(N) = \Omega(N).$$

忽略常数后，这就是论文中 $s(N)t(N) \geq N$ 的含义。它表明平时额外空间和扩缩容临时空间不能同时任意小。若希望稳定存储空间为 $N + O(N^{1/r})$ ，则临时空间至少需要达到 $N + \Omega(N^{1-1/r})$ 。Tarjan 和 Zwick 后续给出的构造正好达到这一量级，因此它在两类空间的 trade-off 上是最优的。

4 本文核心方法

4.1 成员 A：整体构造与技术路线

4.2 整体构造与技术路线：从空间下界到构造目标

在进入本文的具体数据结构构造之前，作者首先从已有的下界出发，重新审视 resizable array 问题中的空间开销。直观上，Brodnik 等人的数据结构已经给出了 $N + O(\sqrt{N})$ 大小的空间使用，这暗示在平时存储空间和扩容过程中的空间开销之间， $\sqrt{N}$ 可能是某种最优平衡。本文的关键观察是：这个下界中混合了两类不同的空间。一种是数据结构在稳定状态下为了存储并访问数组所需要的空间；另一种是 grow 或 shrink 过程中短暂出现的临时空间。作者正是通过区分这两类空间，重新打开了 $N + O(\sqrt{N})$ 这一传统界限。

Theorem 4.1. Brodnik 等人的下界可以表述为：任何维护 resizable array 的数据结构，在某些时刻都必须使用 $N + \Omega(\sqrt{N})$ 空间。这个结论即使只考虑 grow 和 access 操作也成立。

其证明思路可以理解为对 block 数量进行分类讨论。假设经过 N 次 grow 操作后，数组中有 N 个元素，这些元素被存放在若干个动态分配的连续内存块中。设当前一共有 k 个内存块。

第一种情况是 $k \geq \sqrt{N}$ 。由于每一个内存块都必须能够被数据结构访问到，因此每个块至少需要一个指针指向它。这样，仅指针所需要的空间就至少为 k 。而原始数据本身至少需要 N 个空间，所以总空间至少为

$$N + k \geq N + \sqrt{N}.$$

这说明当 block 数量较多时，额外空间主要来自指针开销。也就是说，这一部分对应的是平时为了组织和访问这些 block 而长期存在的存储空间。

第二种情况是 $k < \sqrt{N}$ 。此时， N 个元素被放入少于 $\sqrt{N}$ 个 block 中。根据抽屉原理，必然存在一个 block 的大小大于 $\sqrt{N}$ 。设这个大 block 是在某一次 grow 操作中被申请出来的，当时数组长度为 $N' \leq N$ 。在该 block 刚刚申请出来的瞬间，它还没有真正承载原来的元素，而原来的 N' 个元素仍然存放在旧的 block 中。因此，在这一瞬间，除了旧的 N' 个元素空间之外，还额外出现了一个大小超过 $\sqrt{N'}$ 的新 block，于是当时总空间至少为

$$N' + \Omega(\sqrt{N'}).$$

这说明当 block 数量较少时，必然存在某个很大的 block，而这个大 block 在申请时会产生显著的临时空间。

因此, Theorem 4.1 的两个 case 实际上分别对应两类空间来源: case 1 说明的是平时存储和访问所需的指针空间; case 2 说明的是某一时刻突然申请一个大 block 时产生的临时空间。这也是本文区分平时空间和额外临时空间的原因。

Theorem 4.2. 在 Theorem 4.1 的基础上, 作者进一步给出了更一般的乘积下界。对于任意 $(s(N), t(N))$ -implementation, 都必须满足

$$s(N)t(N) \geq N.$$

其中, $s(N)$ 表示平时存储数组时允许的额外空间, $t(N)$ 表示 grow 或 shrink 过程中允许的临时额外空间。

这个证明与 Theorem 4.1 的思路保持一致, 仍然可以从抽屉原理出发。若数据结构在平时只允许 $N + s(N)$ 的空间, 那么除了 N 个真实元素之外, 最多只能有 $s(N)$ 量级的额外空间用于指针、索引或 block 管理信息。特别地, 由于每个 block 至少需要被某个指针或索引结构访问到, 我们可以认为当前 block 的数量至多为 $s(N)$ 量级。

于是, N 个元素分布在至多 $s(N)$ 个 block 中, 必然存在一个 block 的大小不小于

$$\frac{N}{s(N)}.$$

当这个 block 被申请出来时, 它尚未装入旧元素, 而旧元素仍然存放在原来的 block 中。因此, 在这一瞬间, resize 过程至少需要额外容纳这样一个大小为 $N/s(N)$ 量级的 block。于是有

$$t(N) \geq \frac{N}{s(N)}.$$

进一步得到

$$s(N)t(N) \geq N.$$

这个结论可以看成 Theorem 4.1 的加强版本。因为在 Theorem 4.1 中, 若取 $s(N) = t(N) = \sqrt{N}$, 就正好得到原来的 $N + \Omega(\sqrt{N})$ 下界。更一般地, 如果我们希望平时的额外存储空间达到

$$s(N) = O(N^{1/r}),$$

那么由乘积下界可知, resize 过程中就不可避免地需要

$$t(N) \geq \Omega(N^{1-1/r}).$$

这说明, 如果我们想把稳定状态下的空间压缩到 $N + O(N^{1/r})$, 就必须接受 grow 或 shrink 过程中短暂使用 $N + O(N^{1-1/r})$ 的临时空间。

由下界引出的构造目标。 以上两个定理给出了本文整体构造的理论目标：作者希望围绕乘积下界 $s(N)t(N) \geq N$ 给出相匹配的数据结构。换言之，既然下界已经说明

$$s(N) = O(N^{1/r}) \implies t(N) = \Omega(N^{1-1/r}),$$

那么接下来的目标就是构造一个数据结构，使其真正达到

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}).$$

因此，本文后续构造部分的基本路线可以概括为：先通过下界区分平时空间和临时空间，然后围绕这个下界给出相匹配的结构。构造方面，最基本的 $r = 2$ 情况已经由 Brodnik 等人的 $N + O(\sqrt{N})$ 结构给出；随后作者给出特殊的 $r = 3$ 构造，再进一步推广到一般的 r 。

总结一下：这部分先说明 $s(N)t(N) \geq N$ 的理论最优，再通过 $r = 3$ 的例子展示如何突破稳定状态下的 $N + O(\sqrt{N})$ 空间，最后给出一般 r 的多层 block 构造。

4.3 成员 B：空间优化机制

本节进一步解释上述分层构造中的空间优化机制。核心思想不是消除扩缩容时的数据搬移，而是把空间开销区分为稳定状态下的额外空间和扩缩容过程中的临时空间，并有意识地把较大的空间需求限制在短暂的块合并、拆分和重建过程中[1]。

4.3.1 $r = 3$ 的两级块结构

论文首先讨论一个 $(O(N^{1/3}), O(N^{2/3}))$ -implementation。令参数 B 满足

$$N^{1/3} \leq B \leq 4N^{1/3}.$$

数据结构使用大小为 B^2 的大块存储数组中已经稳定下来的部分，并使用至多 $2B$ 个大小为 B 的小块保存数组末端。所有大块始终保持填满；小块中至多有一个部分填充的块，并可以额外保留一个空块。两类数据块分别由两个索引块管理。

这一设计可以看作对 hashed array tree 的再次分层。如果直接使用大小约为 $N^{2/3}$ 的数据块，最后一个数据块可能只存有少量元素，稳定状态下会浪费 $O(N^{2/3})$ 空间。现在改用大小约为 $N^{1/3}$ 的小块表示末端尚未填满的部分，尾部浪费和索引开销都只需 $O(N^{1/3})$ 空间。

这种压缩并非没有代价。当 $2B$ 个小块均已填满时，结构会将其中 B 个小块的元素复制到一个新申请的、大小为 B^2 的大块中；当小块耗尽而数组继续执行 **Shrink** 时，则需要把一个大块拆成 B 个小块。因此，合并或拆分过程中会短暂使用

$$O(B^2) = O(N^{2/3})$$

的额外空间。由此得到的正是“较小稳定空间、较大临时空间”的折中：平时只保留 $O(N^{1/3})$ 额外空间，而 **resize** 过程允许 $O(N^{2/3})$ 的临时峰值。

4.3.2 一般 r 的分层块结构

对于任意整数 $r \geq 2$ ，第 6 节把两级结构推广为 $r - 1$ 个层级。结构维护参数 B ，使得

$$N^{1/r} \leq B < 4N^{1/r},$$

并使用大小为

$$B, B^2, \dots, B^{r-1}$$

的数据块。大小超过 B 的块始终填满；最低层大小为 B 的块中，至多最后一个没有填满。每种块大小对应一个索引块，因此一共维护 $r - 1$ 个索引块。

各层块的数量可以理解为 N 的一种冗余 base- B 表示。若只支持 **Grow**，每层保留至多 B 个块即可：当某层积累了 B 个大小为 B^i 的满块时，将它们合并成一个大小为 B^{i+1} 的块。为了同时支持 **Shrink**，论文把每层的容量放宽到至多 $2B$ 个块。当最低层没有块可供删除时，结构找到最近的非空高层，并将一个高层块逐级拆成更小的块。这个冗余区间使合并与拆分之间留有缓冲，避免连续操作反复触发相反方向的重构。

稳定状态下，数据块除最低层的最后一个块外均为满块，主要额外开销来自各层索引、计数器和少量未使用位置。每层索引规模为 $O(B)$ ，共有 $r - 1$ 层，故稳定额外空间为

$$O(rB) = O(rN^{1/r}).$$

一次合并所申请的最大新块大小为 B^{r-1} ，因此扩缩容过程中的额外空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

第 6 节由此得到

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation.}$$

4.3.3 访问时间与均摊更新时间

分层并不意味着按下标访问必须逐层查找。论文令 B 为 2 的幂，并把各层块数压缩为机器字中的冗余 base- B 计数器。借助 **msb**、**lsb**、位掩码和移位操作，可以在 word-RAM 模型下以 $O(1)$ 最坏情况时间确定目标元素所在的层、数据块及块内偏移。因此，**Get** 和 **Set** 仍保持常数时间。

合并与拆分虽然可能移动整个块，但其代价可以分摊到一段操作序列上。每个新元素先进入最低层，随后至多沿 $r - 1$ 个层级向上移动；拆分的代价也可以由两次同级拆分之间积累的 **Shrink** 操作支付。因此，**Grow** 和 **Shrink** 的均摊代价为 $O(r)$ 。本章只保留这一结构直觉，完整的 credit 分析和相应下界将在后续理论分析中讨论。

4.3.4 去除稳定空间中的 r 因子

当 r 是常数时, $O(rN^{1/r})$ 与 $O(N^{1/r})$ 在渐近意义下没有区别; 但当 $r = r(N)$ 随 N 增长时, 第 6 节结果中的 r 因子不能忽略。第 7 节为此提出 data structuring transformation。它针对一类 standard implementation: 临时空间峰值主要出现在申请新块, 并按顺序把若干旧块内容复制到新块的过程中。

设一个标准结构的参数原本为

$$(s(N), O(N)),$$

并给定非递减阈值函数 $t(N)$ 。当它试图申请长度超过 $t(N)$ 的新块时, 不再申请单个完整大块, 而是顺序申请若干个大小至多为 $t(N)$ 的小块, 并保存指向这些小块的额外指针。这样, 任一时刻新申请而尚未填满的空间被控制在 $O(t(N))$; 另一方面, 拆分大块所需的额外指针数为 $O(N/t(N))$ 。Lemma 7.2 因而把原结构转换为

$$\left(s(N) + O\left(\frac{N}{t(N)}\right), s(N) + O(t(N))\right)\text{-implementation,}$$

同时不增加访问操作的渐近最坏情况时间, 也不增加扩缩容操作的渐近均摊时间。

为了得到最终界, 论文取

$$t(N) = N^{1-1/r},$$

并把该转换应用于第 6 节中参数为 $2r$ 的标准结构。输入结构的稳定额外空间为

$$O(rN^{1/(2r)}).$$

Theorem 7.3 所处理的参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}$$

在此范围内, 上述空间项以及新增的

$$O(N/t(N)) = O(N^{1/r})$$

指针空间都可被 $O(N^{1/r})$ 吸收。最终得到

$$(O(N^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

并继续支持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。这个转换说明, 论文的最终空间界并非只来自增加层数, 而是来自“分层表示”与“限制单次物理分配规模”两步优化的结合。

5 正确性证明与理论分析

5.1 成员 A：上界分析

待补充。

5.2 成员 B：下界分析

待补充。

6 技术直觉与方法评价

6.1 成员 A：技术直觉与示例解释

待补充。

6.2 成员 B：创新点、局限性与后续影响

待补充。

7 总结

7.1 成员 A：总结

待补充。

7.2 成员 B：参考文献与过程说明

待补充。

参考文献

- [1] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays, 2023.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Edward Sitarski. Algorithm alley: Hats: Hashed array trees. *Dr. Dobbs's Journal of Software Tools*, 21(9):107, 1996.

- [4] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Proceedings of the 6th International Workshop on Algorithms and Data Structures (WADS)*, pages 37–48, 1999.